

SQL Server Practice Workbook

200+ Intermediate Tasks & Solutions — SalesDB Dataset

217 exercises across 14 topic areas

This workbook contains **217 intermediate-level T-SQL exercises** built on the **SalesDB** database (Sales.Customers, Sales.Employees, Sales.Products, Sales.Orders, Sales.OrdersArchive). Topics include filtering, aggregation, joins, subqueries, window functions, date/string functions, CASE expressions, set operations, CTEs (including recursion), self-joins, NULL handling, and TOP-N/pagination. Each task states a clear objective, followed by a complete, ready-to-run T-SQL solution. Try solving each task yourself before checking the provided solution.

Database Schema Overview

Table	Key Columns
Sales.Customers	CustomerID (PK), FirstName, LastName, Country, Score
Sales.Employees	EmployeeID (PK), FirstName, LastName, Department, BirthDate, Gender, Salary, ManagerID
Sales.Products	ProductID (PK), Product, Category, Price
Sales.Orders	OrderID (PK), ProductID, CustomerID, SalesPersonID, OrderDate, ShipDate, OrderStatus, ShipAddress, BillAddress, Quantity, Sales, CreationTime
Sales.OrdersArchive	Same columns as Sales.Orders (historical/archived orders, no PK)

Table of Contents

Section	# Tasks
A. Filtering & WHERE Clauses	20
B. Aggregation & GROUP BY	26
C. Joins	26
D. Subqueries	20
E. Window Functions	25
F. Date & Time Functions	15
G. String Functions	15
H. CASE Expressions	15
I. Set Operations (UNION / INTERSECT / EXCEPT)	10
J. Common Table Expressions (CTEs)	10
K. Self Joins	5
L. NULL Handling	10
M. TOP-N & Pagination	10
N. Miscellaneous	10
TOTAL	217

A. Filtering & WHERE Clauses

20 tasks in this section

Task 1

List all customers located in 'Germany'.

SOLUTION

```
SELECT * FROM Sales.Customers
WHERE Country = 'Germany';
```

Task 2

Find all customers whose Score is greater than 500.

SOLUTION

```
SELECT * FROM Sales.Customers
WHERE Score > 500;
```

Task 3

Find all customers with a NULL Score.

SOLUTION

```
SELECT * FROM Sales.Customers
WHERE Score IS NULL;
```

Task 4

Find all employees in the 'Sales' department.

SOLUTION

```
SELECT * FROM Sales.Employees
WHERE Department = 'Sales';
```

Task 5

Find all employees whose Salary is between 4000 and 7000 (inclusive).

SOLUTION

```
SELECT * FROM Sales.Employees
WHERE Salary BETWEEN 4000 AND 7000;
```

Task 6

Find all female employees ('F') who do not have a manager assigned.

SOLUTION

```
SELECT * FROM Sales.Employees
WHERE Gender = 'F' AND ManagerID IS NULL;
```

Task 7

Find all products priced above 100.

SOLUTION

```
SELECT * FROM Sales.Products
WHERE Price > 100;
```

Task 8

Find all products in the 'Bikes' or 'Accessories' categories.

SOLUTION

```
SELECT * FROM Sales.Products
WHERE Category IN ('Bikes', 'Accessories');
```

Task 9

Find all products whose name starts with the letter 'M'.

SOLUTION

```
SELECT * FROM Sales.Products
WHERE Product LIKE 'M%';
```

Task 10

Find all orders with a Quantity greater than 5.

SOLUTION

```
SELECT * FROM Sales.Orders
WHERE Quantity > 5;
```

Task 11

Find all orders placed in the year 2023.

SOLUTION

```
SELECT * FROM Sales.Orders
WHERE YEAR(OrderDate) = 2023;
```

Task 12

Find all orders with the status 'Delivered'.

SOLUTION

```
SELECT * FROM Sales.Orders
WHERE OrderStatus = 'Delivered';
```

Task 13

Find all orders that have not yet been shipped (ShipDate is NULL).

SOLUTION

```
SELECT * FROM Sales.Orders
WHERE ShipDate IS NULL;
```

Task 14

Find all orders where the ShipAddress is different from the BillAddress.

SOLUTION

```
SELECT * FROM Sales.Orders
WHERE ShipAddress <> BillAddress;
```

Task 15

Find all orders with Sales greater than 1000 placed by CustomerID 5, 8, or 11.

SOLUTION

```
SELECT * FROM Sales.Orders
WHERE Sales > 1000
      AND CustomerID IN (5, 8, 11);
```

Task 16

Find all customers whose LastName contains the substring 'an'.

SOLUTION

```
SELECT * FROM Sales.Customers
WHERE LastName LIKE '%an%';
```

Task 17

Find all employees born before January 1, 1990.

SOLUTION

```
SELECT * FROM Sales.Employees
WHERE BirthDate < '1990-01-01';
```

Task 18

Find all orders NOT placed by CustomerID 1.

SOLUTION

```
SELECT * FROM Sales.Orders
WHERE CustomerID <> 1;
```

Task 19

Find all products with a price that is NOT between 20 and 50.

SOLUTION

```
SELECT * FROM Sales.Products
WHERE Price NOT BETWEEN 20 AND 50;
```

Task 20

Find all orders placed in the first quarter (Jan-Mar) of any year.

SOLUTION

```
SELECT * FROM Sales.Orders
WHERE MONTH(OrderDate) IN (1, 2, 3);
```

B. Aggregation & GROUP BY

26 tasks in this section

Task 21

Count the total number of customers.

SOLUTION

```
SELECT COUNT(*) AS TotalCustomers FROM Sales.Customers;
```

Task 22

Count the number of customers per country.

SOLUTION

```
SELECT Country, COUNT(*) AS CustomerCount
FROM Sales.Customers
GROUP BY Country
ORDER BY CustomerCount DESC;
```

Task 23

Find the average Score of customers per country.

SOLUTION

```
SELECT Country, AVG(Score) AS AvgScore
FROM Sales.Customers
GROUP BY Country
ORDER BY AvgScore DESC;
```

Task 24

Find the highest and lowest Score in the Customers table.

SOLUTION

```
SELECT MAX(Score) AS HighestScore, MIN(Score) AS LowestScore
FROM Sales.Customers;
```

Task 25

Count the number of employees in each department.

SOLUTION

```
SELECT Department, COUNT(*) AS EmployeeCount
FROM Sales.Employees
GROUP BY Department;
```

Task 26

Find the total and average salary per department.

SOLUTION

```
SELECT
    Department,
    SUM(Salary) AS TotalSalary,
    AVG(Salary) AS AvgSalary
FROM Sales.Employees
GROUP BY Department;
```

Task 27

Find departments that have more than 3 employees.

SOLUTION

```
SELECT Department, COUNT(*) AS EmployeeCount
FROM Sales.Employees
GROUP BY Department
HAVING COUNT(*) > 3;
```

Task 28

Find the number of products in each category, sorted from most to fewest.

SOLUTION

```
SELECT Category, COUNT(*) AS ProductCount
FROM Sales.Products
GROUP BY Category
ORDER BY ProductCount DESC;
```

Task 29

Find the average price of products per category.

SOLUTION

```
SELECT Category, AVG(Price) AS AvgPrice
FROM Sales.Products
GROUP BY Category;
```

Task 30

Find categories whose average product price exceeds 100.

SOLUTION

```
SELECT Category, AVG(Price) AS AvgPrice
FROM Sales.Products
GROUP BY Category
HAVING AVG(Price) > 100;
```

Task 31

Calculate the total Sales amount for all orders.

SOLUTION

```
SELECT SUM(Sales) AS TotalSales FROM Sales.Orders;
```

Task 32

Calculate the total Quantity ordered per ProductID.

SOLUTION

```
SELECT ProductID, SUM(Quantity) AS TotalQuantity
FROM Sales.Orders
GROUP BY ProductID
ORDER BY TotalQuantity DESC;
```

Task 33

Find the number of orders placed by each customer.

SOLUTION

```
SELECT CustomerID, COUNT(*) AS OrderCount
FROM Sales.Orders
GROUP BY CustomerID;
```

Task 34

Find customers who have placed more than 3 orders.

SOLUTION

```
SELECT CustomerID, COUNT(*) AS OrderCount
FROM Sales.Orders
GROUP BY CustomerID
HAVING COUNT(*) > 3;
```

Task 35

Find the total Sales generated by each SalesPersonID.

SOLUTION

```
SELECT SalesPersonID, SUM(Sales) AS TotalSales
FROM Sales.Orders
GROUP BY SalesPersonID
ORDER BY TotalSales DESC;
```

Task 36

Find the average order value (Sales) per OrderStatus.

SOLUTION

```
SELECT OrderStatus, AVG(Sales) AS AvgSales
FROM Sales.Orders
GROUP BY OrderStatus;
```

Task 37

Find the total number of orders and total sales per year.

SOLUTION

```
SELECT
    YEAR(OrderDate) AS OrderYear,
    COUNT(*) AS TotalOrders,
    SUM(Sales) AS TotalSales
FROM Sales.Orders
GROUP BY YEAR(OrderDate)
ORDER BY OrderYear;
```

Task 38

Find the highest single Sales value recorded for each ProductID.

SOLUTION

```
SELECT ProductID, MAX(Sales) AS HighestSale
FROM Sales.Orders
GROUP BY ProductID;
```

Task 39

Find products that generated a total Sales figure greater than 5000.

SOLUTION

```
SELECT ProductID, SUM(Sales) AS TotalSales
FROM Sales.Orders
GROUP BY ProductID
HAVING SUM(Sales) > 5000;
```

Task 40

Count how many distinct customers placed at least one order.

SOLUTION

```
SELECT COUNT(DISTINCT CustomerID) AS DistinctCustomers FROM Sales.Orders;
```

Task 41

Count how many distinct products were ordered.

SOLUTION

```
SELECT COUNT(DISTINCT ProductID) AS DistinctProductsOrdered FROM Sales.Orders;
```

Task 42

Find the number of orders per OrderStatus, sorted descending.

SOLUTION

```
SELECT OrderStatus, COUNT(*) AS OrderCount
FROM Sales.Orders
GROUP BY OrderStatus
ORDER BY OrderCount DESC;
```

Task 43

Find the total Quantity and total Sales per month across all years.

SOLUTION

```
SELECT
    MONTH(OrderDate) AS OrderMonth,
    SUM(Quantity) AS TotalQuantity,
    SUM(Sales) AS TotalSales
FROM Sales.Orders
GROUP BY MONTH(OrderDate)
ORDER BY OrderMonth;
```

Task 44

Find the minimum and maximum Salary for each Gender.

SOLUTION

```
SELECT Gender, MIN(Salary) AS MinSalary, MAX(Salary) AS MaxSalary
FROM Sales.Employees
GROUP BY Gender;
```

Task 45

Find the average Quantity per order for each SalesPersonID, rounded to 2 decimals.

SOLUTION

```
SELECT SalesPersonID, ROUND(AVG(CAST(Quantity AS FLOAT)), 2) AS AvgQuantity
FROM Sales.Orders
GROUP BY SalesPersonID;
```

Task 46

Find customers whose combined order Sales exceed their own Score value.

SOLUTION

```
SELECT
    c.CustomerID,
    c.Score,
    SUM(o.Sales) AS TotalSales
FROM Sales.Customers AS c
JOIN Sales.Orders AS o
    ON o.CustomerID = c.CustomerID
GROUP BY c.CustomerID, c.Score
HAVING SUM(o.Sales) > c.Score;
```

C. Joins

26 tasks in this section

Task 47

List each order with the customer's full name and the product name.

SOLUTION

```
SELECT
    o.OrderID,
    c.FirstName + ' ' + c.LastName AS CustomerName,
    p.Product,
    o.Quantity,
    o.Sales
FROM Sales.Orders AS o
JOIN Sales.Customers AS c ON c.CustomerID = o.CustomerID
JOIN Sales.Products AS p ON p.ProductID = o.ProductID;
```

Task 48

List all customers and, if they exist, their orders (include customers with no orders).

SOLUTION

```
SELECT
    c.CustomerID,
    c.FirstName + ' ' + c.LastName AS CustomerName,
    o.OrderID,
    o.Sales
FROM Sales.Customers AS c
LEFT JOIN Sales.Orders AS o
    ON o.CustomerID = c.CustomerID;
```

Task 49

List all products and, if they exist, the orders that reference them (include unordered products).

SOLUTION

```
SELECT
    p.ProductID,
    p.Product,
    o.OrderID,
    o.Quantity
FROM Sales.Products AS p
LEFT JOIN Sales.Orders AS o
    ON o.ProductID = p.ProductID;
```

Task 50

Find all orders along with the salesperson's name (join Orders to Employees on SalesPersonID).

SOLUTION

```
SELECT
    o.OrderID,
    e.FirstName + ' ' + e.LastName AS SalesPersonName,
    o.Sales
FROM Sales.Orders AS o
JOIN Sales.Employees AS e
    ON e.EmployeeID = o.SalesPersonID;
```

Task 51

Find customers who have never placed an order (RIGHT/LEFT JOIN style).

SOLUTION

```
SELECT c.CustomerID, c.FirstName, c.LastName
FROM Sales.Customers AS c
LEFT JOIN Sales.Orders AS o
      ON o.CustomerID = c.CustomerID
WHERE o.OrderID IS NULL;
```

Task 52

Find all orders that reference a CustomerID that does not exist in the Customers table (orphan orders).

SOLUTION

```
SELECT o.*
FROM Sales.Orders AS o
LEFT JOIN Sales.Customers AS c
      ON c.CustomerID = o.CustomerID
WHERE c.CustomerID IS NULL;
```

Task 53

List every customer-product combination that resulted in at least one order, with total quantity.

SOLUTION

```
SELECT
    c.CustomerID,
    c.FirstName + ' ' + c.LastName AS CustomerName,
    p.Product,
    SUM(o.Quantity) AS TotalQuantity
FROM Sales.Orders AS o
JOIN Sales.Customers AS c ON c.CustomerID = o.CustomerID
JOIN Sales.Products AS p ON p.ProductID = o.ProductID
GROUP BY c.CustomerID, c.FirstName, c.LastName, p.Product;
```

Task 54

Show a full outer join between Customers and Orders to spot customers without orders and orders without a valid customer.

SOLUTION

```
SELECT
    c.CustomerID,
    o.OrderID
FROM Sales.Customers AS c
FULL OUTER JOIN Sales.Orders AS o
      ON o.CustomerID = c.CustomerID
WHERE c.CustomerID IS NULL OR o.OrderID IS NULL;
```

Task 55

List each order with the product's category and price, and compute Sales minus (Price * Quantity) as a price variance.

SOLUTION

```
SELECT
    o.OrderID,
    p.Product,
    p.Category,
    p.Price,
    o.Quantity,
    o.Sales,
    o.Sales - (p.Price * o.Quantity) AS PriceVariance
FROM Sales.Orders AS o
JOIN Sales.Products AS p
    ON p.ProductID = o.ProductID;
```

Task 56

For each employee, list the total Sales they generated as a salesperson (LEFT JOIN so 0-order employees appear too).

SOLUTION

```
SELECT
    e.EmployeeID,
    e.FirstName + ' ' + e.LastName AS EmployeeName,
    ISNULL(SUM(o.Sales), 0) AS TotalSales
FROM Sales.Employees AS e
LEFT JOIN Sales.Orders AS o
    ON o.SalesPersonID = e.EmployeeID
GROUP BY e.EmployeeID, e.FirstName, e.LastName;
```

Task 57

List each order together with the employee's department who sold it.

SOLUTION

```
SELECT
    o.OrderID,
    e.Department,
    o.Sales
FROM Sales.Orders AS o
JOIN Sales.Employees AS e
    ON e.EmployeeID = o.SalesPersonID;
```

Task 58

List all orders for customers located in 'USA', showing the product name too.

SOLUTION

```
SELECT
    o.OrderID,
    c.FirstName + ' ' + c.LastName AS CustomerName,
    p.Product,
    o.Sales
FROM Sales.Orders AS o
JOIN Sales.Customers AS c ON c.CustomerID = o.CustomerID
JOIN Sales.Products AS p ON p.ProductID = o.ProductID
WHERE c.Country = 'USA';
```

Task 59

Find the total Sales per product category (join Orders to Products, then group).

SOLUTION

```
SELECT
    p.Category,
    SUM(o.Sales) AS TotalSales
FROM Sales.Orders AS o
JOIN Sales.Products AS p
    ON p.ProductID = o.ProductID
GROUP BY p.Category
ORDER BY TotalSales DESC;
```

Task 60

Find the top customer (by total Sales) for each country.

SOLUTION

```
WITH CustomerSales AS (
    SELECT
        c.CustomerID,
        c.Country,
        c.FirstName + ' ' + c.LastName AS CustomerName,
        SUM(o.Sales) AS TotalSales,
        ROW_NUMBER() OVER (PARTITION BY c.Country ORDER BY SUM(o.Sales) DESC) AS rn
    FROM Sales.Customers AS c
    JOIN Sales.Orders AS o ON o.CustomerID = c.CustomerID
    GROUP BY c.CustomerID, c.Country, c.FirstName, c.LastName
)
SELECT CustomerID, Country, CustomerName, TotalSales
FROM CustomerSales
WHERE rn = 1;
```

Task 61

Join Orders to both Customers and Products, and show only orders where the customer's Score is above 400 and the product Category is 'Bikes'.

SOLUTION

```
SELECT
    o.OrderID,
    c.FirstName + ' ' + c.LastName AS CustomerName,
    c.Score,
    p.Product,
    p.Category
FROM Sales.Orders AS o
JOIN Sales.Customers AS c ON c.CustomerID = o.CustomerID
JOIN Sales.Products AS p ON p.ProductID = o.ProductID
WHERE c.Score > 400 AND p.Category = 'Bikes';
```

Task 62

List employees together with their manager's name (self-referential join via LEFT JOIN to include employees with no manager).

SOLUTION

```
SELECT
    e.EmployeeID,
    e.FirstName + ' ' + e.LastName AS EmployeeName,
    m.FirstName + ' ' + m.LastName AS ManagerName
FROM Sales.Employees AS e
LEFT JOIN Sales.Employees AS m
    ON e.ManagerID = m.EmployeeID;
```

Task 63

Find pairs of customers from the same country (excluding pairing a customer with themselves).

SOLUTION

```
SELECT
    c1.CustomerID AS Customer1ID,
    c2.CustomerID AS Customer2ID,
    c1.Country
FROM Sales.Customers AS c1
JOIN Sales.Customers AS c2
    ON c1.Country = c2.Country
    AND c1.CustomerID < c2.CustomerID;
```

Task 64

Join Orders with OrdersArchive to list orders present in the archive along with their live-table Sales value.

SOLUTION

```
SELECT
    o.OrderID,
    o.Sales AS LiveSales,
    a.Sales AS ArchivedSales
FROM Sales.Orders AS o
JOIN Sales.OrdersArchive AS a
    ON a.OrderID = o.OrderID;
```

Task 65

Find products that were ordered by customers from more than one distinct country.

SOLUTION

```
SELECT
    p.ProductID,
    p.Product,
    COUNT(DISTINCT c.Country) AS DistinctCountries
FROM Sales.Orders AS o
JOIN Sales.Products AS p ON p.ProductID = o.ProductID
JOIN Sales.Customers AS c ON c.CustomerID = o.CustomerID
GROUP BY p.ProductID, p.Product
HAVING COUNT(DISTINCT c.Country) > 1;
```

Task 66

List each salesperson's employee record together with the number of distinct customers they've sold to.

SOLUTION

```
SELECT
    e.EmployeeID,
    e.FirstName + ' ' + e.LastName AS SalesPersonName,
    COUNT(DISTINCT o.CustomerID) AS DistinctCustomersServed
FROM Sales.Employees AS e
JOIN Sales.Orders AS o
    ON o.SalesPersonID = e.EmployeeID
GROUP BY e.EmployeeID, e.FirstName, e.LastName;
```

Task 67

Find orders where the ordering customer's Country differs from 'Germany' but the salesperson's Department is 'Sales'.

SOLUTION

```
SELECT
    o.OrderID,
    c.Country AS CustomerCountry,
    e.Department AS SalesDepartment
FROM Sales.Orders AS o
JOIN Sales.Customers AS c ON c.CustomerID = o.CustomerID
JOIN Sales.Employees AS e ON e.EmployeeID = o.SalesPersonID
WHERE c.Country <> 'Germany' AND e.Department = 'Sales';
```

Task 68

List the three tables Customers, Orders, Products together, restricted to Delivered orders only, showing customer, product, and sales.

SOLUTION

```
SELECT
    c.FirstName + ' ' + c.LastName AS CustomerName,
    p.Product,
    o.Sales,
    o.OrderStatus
FROM Sales.Orders AS o
JOIN Sales.Customers AS c ON c.CustomerID = o.CustomerID
JOIN Sales.Products AS p ON p.ProductID = o.ProductID
WHERE o.OrderStatus = 'Delivered';
```

Task 69

Find the average product Price for products that have been ordered at least once (join to Orders, distinct products).

SOLUTION

```
SELECT AVG(p.Price) AS AvgOrderedProductPrice
FROM Sales.Products AS p
WHERE EXISTS (
    SELECT 1 FROM Sales.Orders AS o WHERE o.ProductID = p.ProductID
);
```

Task 70

List customers and their most recent order date (LEFT JOIN + GROUP BY, include customers with no orders as NULL).

SOLUTION

```
SELECT
    c.CustomerID,
    c.FirstName + ' ' + c.LastName AS CustomerName,
    MAX(o.OrderDate) AS LastOrderDate
FROM Sales.Customers AS c
LEFT JOIN Sales.Orders AS o
    ON o.CustomerID = c.CustomerID
GROUP BY c.CustomerID, c.FirstName, c.LastName;
```

Task 71

Find employees who have sold products in the 'Accessories' category, listing employee name and total quantity sold in that category.

SOLUTION

```
SELECT
    e.FirstName + ' ' + e.LastName AS EmployeeName,
    SUM(o.Quantity) AS AccessoriesQuantitySold
FROM Sales.Orders AS o
JOIN Sales.Employees AS e ON e.EmployeeID = o.SalesPersonID
JOIN Sales.Products AS p ON p.ProductID = o.ProductID
WHERE p.Category = 'Accessories'
GROUP BY e.FirstName, e.LastName;
```

Task 72

Find customers whose ShipAddress on at least one order matches another customer's BillAddress (possible shared address, join Orders to itself via address).

SOLUTION

```
SELECT DISTINCT
    o1.CustomerID AS Customer1,
    o2.CustomerID AS Customer2,
    o1.ShipAddress
FROM Sales.Orders AS o1
JOIN Sales.Orders AS o2
    ON o1.ShipAddress = o2.BillAddress
    AND o1.CustomerID <> o2.CustomerID;
```

D. Subqueries

20 tasks in this section

Task 73

Find customers whose Score is higher than the average Score (scalar subquery).

SOLUTION

```
SELECT *
FROM Sales.Customers
WHERE Score > (SELECT AVG(Score) FROM Sales.Customers);
```

Task 74

Find the product(s) with the highest Price using a subquery.

SOLUTION

```
SELECT *
FROM Sales.Products
WHERE Price = (SELECT MAX(Price) FROM Sales.Products);
```

Task 75

Find employees who earn more than the average salary of their own department (correlated subquery).

SOLUTION

```
SELECT e1.*
FROM Sales.Employees AS e1
WHERE e1.Salary > (
    SELECT AVG(e2.Salary)
    FROM Sales.Employees AS e2
    WHERE e2.Department = e1.Department
);
```

Task 76

Find customers who have placed at least one order (using EXISTS).

SOLUTION

```
SELECT *
FROM Sales.Customers AS c
WHERE EXISTS (
    SELECT 1 FROM Sales.Orders AS o WHERE o.CustomerID = c.CustomerID
);
```

Task 77

Find customers who have never placed an order (using NOT EXISTS).

SOLUTION

```
SELECT *
FROM Sales.Customers AS c
WHERE NOT EXISTS (
    SELECT 1 FROM Sales.Orders AS o WHERE o.CustomerID = c.CustomerID
);
```

Task 78

Find products whose ProductID appears in at least one order with Quantity greater than 10 (using IN).

SOLUTION

```
SELECT *
FROM Sales.Products
WHERE ProductID IN (
    SELECT ProductID FROM Sales.Orders WHERE Quantity > 10
);
```

Task 79

Find products that have never appeared in any order (using NOT IN).

SOLUTION

```
SELECT *
FROM Sales.Products
WHERE ProductID NOT IN (
    SELECT ProductID FROM Sales.Orders WHERE ProductID IS NOT NULL
);
```

Task 80

Find the customer(s) with the single highest total Sales (nested aggregation subquery).

SOLUTION

```
SELECT TOP 1 WITH TIES
    CustomerID,
    SUM(Sales) AS TotalSales
FROM Sales.Orders
GROUP BY CustomerID
ORDER BY SUM(Sales) DESC;
```

Task 81

Find employees whose Salary is the maximum within their department (correlated subquery).

SOLUTION

```
SELECT e1.*
FROM Sales.Employees AS e1
WHERE e1.Salary = (
    SELECT MAX(e2.Salary)
    FROM Sales.Employees AS e2
    WHERE e2.Department = e1.Department
);
```

Task 82

Find orders whose Sales value is above the overall average Sales (subquery in WHERE).

SOLUTION

```
SELECT *
FROM Sales.Orders
WHERE Sales > (SELECT AVG(Sales) FROM Sales.Orders);
```

Task 83

For each order, show how much the Sales value differs from the average Sales of that same product (correlated subquery in SELECT).

SOLUTION

```
SELECT
    o.OrderID,
    o.ProductID,
    o.Sales,
    o.Sales - (
        SELECT AVG(o2.Sales)
        FROM Sales.Orders AS o2
        WHERE o2.ProductID = o.ProductID
    ) AS DiffFromProductAvg
FROM Sales.Orders AS o;
```

Task 84

Find the second-highest Salary among all employees using a subquery.

SOLUTION

```
SELECT MAX(Salary) AS SecondHighestSalary
FROM Sales.Employees
WHERE Salary < (SELECT MAX(Salary) FROM Sales.Employees);
```

Task 85

Find customers located in a country where the average Score (across that country) is below 300.

SOLUTION

```
SELECT *
FROM Sales.Customers AS c
WHERE c.Country IN (
    SELECT Country
    FROM Sales.Customers
    GROUP BY Country
    HAVING AVG(Score) < 300
);
```

Task 86

List orders placed by the customer who has the highest Score (subquery for the top customer).

SOLUTION

```
SELECT *
FROM Sales.Orders
WHERE CustomerID = (
    SELECT TOP 1 CustomerID
    FROM Sales.Customers
    ORDER BY Score DESC
);
```

Task 87

Find products priced above the average price within their own category (correlated subquery).

SOLUTION

```
SELECT p1.*
FROM Sales.Products AS p1
WHERE p1.Price > (
    SELECT AVG(p2.Price)
    FROM Sales.Products AS p2
    WHERE p2.Category = p1.Category
);
```

Task 88

Find the employee(s) who are NOT a manager of anyone (not referenced by any ManagerID).

SOLUTION

```
SELECT *
FROM Sales.Employees AS e
WHERE e.EmployeeID NOT IN (
    SELECT ManagerID FROM Sales.Employees WHERE ManagerID IS NOT NULL
);
```

Task 89

Find orders whose ProductID belongs to the category with the highest average Price (nested subqueries).

SOLUTION

```
SELECT *
FROM Sales.Orders
WHERE ProductID IN (
    SELECT ProductID
    FROM Sales.Products
    WHERE Category = (
        SELECT TOP 1 Category
        FROM Sales.Products
        GROUP BY Category
        ORDER BY AVG(Price) DESC
    )
);
```

Task 90

Find customers whose total order Sales is below the overall average total Sales per customer.

SOLUTION

```
SELECT CustomerID, SUM(Sales) AS TotalSales
FROM Sales.Orders
GROUP BY CustomerID
HAVING SUM(Sales) < (
    SELECT AVG(CustomerTotal.TotalSales)
    FROM (
        SELECT SUM(Sales) AS TotalSales
        FROM Sales.Orders
        GROUP BY CustomerID
    ) AS CustomerTotal
);
```

Task 91

Find the most recent order for each customer using a correlated subquery.

SOLUTION

```
SELECT o1.*
FROM Sales.Orders AS o1
WHERE o1.OrderDate = (
    SELECT MAX(o2.OrderDate)
    FROM Sales.Orders AS o2
    WHERE o2.CustomerID = o1.CustomerID
);
```

Task 92

Find employees who manage more than 2 direct reports (subquery + HAVING).

SOLUTION

```
SELECT ManagerID, COUNT(*) AS DirectReports
FROM Sales.Employees
WHERE ManagerID IS NOT NULL
GROUP BY ManagerID
HAVING COUNT(*) > 2;
```

E. Window Functions

25 tasks in this section

Task 93

Assign a unique sequential number to each order, ordered by OrderDate (ROW_NUMBER).

SOLUTION

```
SELECT
    OrderID,
    OrderDate,
    ROW_NUMBER() OVER (ORDER BY OrderDate) AS RowNum
FROM Sales.Orders;
```

Task 94

Rank all employees by Salary in descending order, company-wide (RANK).

SOLUTION

```
SELECT
    EmployeeID,
    FirstName + ' ' + LastName AS EmployeeName,
    Salary,
    RANK() OVER (ORDER BY Salary DESC) AS SalaryRank
FROM Sales.Employees;
```

Task 95

Rank employees by Salary within each Department using DENSE_RANK.

SOLUTION

```
SELECT
    EmployeeID,
    Department,
    Salary,
    DENSE_RANK() OVER (PARTITION BY Department ORDER BY Salary DESC) AS DeptSalaryRank
FROM Sales.Employees;
```

Task 96

Number each customer's orders in chronological order (ROW_NUMBER partitioned by customer).

SOLUTION

```
SELECT
    CustomerID,
    OrderID,
    OrderDate,
    ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY OrderDate) AS OrderSequence
FROM Sales.Orders;
```

Task 97

Find each customer's first order (using ROW_NUMBER = 1 in a CTE).

SOLUTION

```
WITH NumberedOrders AS (  
    SELECT  
        *,  
        ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY OrderDate) AS rn  
    FROM Sales.Orders  
)  
SELECT *  
FROM NumberedOrders  
WHERE rn = 1;
```

Task 98

Calculate a running total of Sales across all orders, ordered by OrderDate.

SOLUTION

```
SELECT  
    OrderID,  
    OrderDate,  
    Sales,  
    SUM(Sales) OVER (ORDER BY OrderDate ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS RunningTotal  
FROM Sales.Orders;
```

Task 99

Calculate a 3-order moving average of Sales per customer, ordered by OrderDate.

SOLUTION

```
SELECT  
    CustomerID,  
    OrderID,  
    OrderDate,  
    Sales,  
    AVG(Sales) OVER (  
        PARTITION BY CustomerID  
        ORDER BY OrderDate  
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW  
    ) AS MovingAvg3  
FROM Sales.Orders;
```

Task 100

For each order, show the Sales value of the customer's previous order (LAG).

SOLUTION

```
SELECT  
    CustomerID,  
    OrderID,  
    OrderDate,  
    Sales,  
    LAG(Sales) OVER (PARTITION BY CustomerID ORDER BY OrderDate) AS PreviousOrderSales  
FROM Sales.Orders;
```

Task 101

For each order, show the Sales value of the customer's next order (LEAD).

SOLUTION

```
SELECT
    CustomerID,
    OrderID,
    OrderDate,
    Sales,
    LEAD(Sales) OVER (PARTITION BY CustomerID ORDER BY OrderDate) AS NextOrderSales
FROM Sales.Orders;
```

Task 102

Calculate the change in Sales compared to the customer's previous order (LAG + subtraction).

SOLUTION

```
SELECT
    CustomerID,
    OrderID,
    OrderDate,
    Sales,
    Sales - LAG(Sales) OVER (PARTITION BY CustomerID ORDER BY OrderDate) AS SalesChange
FROM Sales.Orders;
```

Task 103

Split all customers into 4 equally sized groups (quartiles) based on Score (NTILE).

SOLUTION

```
SELECT
    CustomerID,
    Score,
    NTILE(4) OVER (ORDER BY Score DESC) AS ScoreQuartile
FROM Sales.Customers;
```

Task 104

Show each order's Sales value alongside the total Sales for that customer (SUM OVER PARTITION).

SOLUTION

```
SELECT
    CustomerID,
    OrderID,
    Sales,
    SUM(Sales) OVER (PARTITION BY CustomerID) AS CustomerTotalSales
FROM Sales.Orders;
```

Task 105

Show each order's Sales as a percentage of its customer's total Sales.

SOLUTION

```
SELECT
    CustomerID,
    OrderID,
    Sales,
    CAST(Sales AS FLOAT) / SUM(Sales) OVER (PARTITION BY CustomerID) * 100 AS PctOfCustomerTotal
FROM Sales.Orders;
```

Task 106

Find the first and last order date for each customer using FIRST_VALUE and LAST_VALUE.

SOLUTION

```
SELECT DISTINCT
    CustomerID,
    FIRST_VALUE(OrderDate) OVER (
        PARTITION BY CustomerID ORDER BY OrderDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS FirstOrderDate,
    LAST_VALUE(OrderDate) OVER (
        PARTITION BY CustomerID ORDER BY OrderDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS LastOrderDate
FROM Sales.Orders;
```

Task 107

Find the top 5 orders by Sales value using ROW_NUMBER inside a CTE.

SOLUTION

```
WITH RankedOrders AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY Sales DESC) AS rn
    FROM Sales.Orders
)
SELECT *
FROM RankedOrders
WHERE rn <= 5;
```

Task 108

For each product, rank its orders by Quantity in descending order and keep only rank 1 (top order per product).

SOLUTION

```
WITH RankedByProduct AS (
    SELECT
        *,
        RANK() OVER (PARTITION BY ProductID ORDER BY Quantity DESC) AS QtyRank
    FROM Sales.Orders
)
SELECT *
FROM RankedByProduct
WHERE QtyRank = 1;
```

Task 109

Calculate each employee's Salary as a percentage of the total salary paid in their department.

SOLUTION

```
SELECT
    EmployeeID,
    Department,
    Salary,
    CAST(Salary AS FLOAT) / SUM(Salary) OVER (PARTITION BY Department) * 100 AS PctOfDeptSalary
FROM Sales.Employees;
```

Task 110

Find the difference between each employee's Salary and the highest Salary in their department.

SOLUTION

```
SELECT
    EmployeeID,
    Department,
    Salary,
    MAX(Salary) OVER (PARTITION BY Department) - Salary AS GapFromTopEarner
FROM Sales.Employees;
```

Task 111

Number products within each Category ordered by Price descending, and flag the most expensive one per category.

SOLUTION

```
SELECT
    ProductID,
    Category,
    Price,
    ROW_NUMBER() OVER (PARTITION BY Category ORDER BY Price DESC) AS PriceRank,
    CASE WHEN ROW_NUMBER() OVER (PARTITION BY Category ORDER BY Price DESC) = 1
        THEN 'Most Expensive' ELSE '' END AS Flag
FROM Sales.Products;
```

Task 112

Calculate the cumulative count of orders per customer over time (COUNT OVER).

SOLUTION

```
SELECT
    CustomerID,
    OrderID,
    OrderDate,
    COUNT(*) OVER (
        PARTITION BY CustomerID
        ORDER BY OrderDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS OrdersSoFar
FROM Sales.Orders;
```

Task 113

Find the average Sales per SalesPersonID and show how each of their individual orders compares to it.

SOLUTION

```
SELECT
    SalesPersonID,
    OrderID,
    Sales,
    AVG(Sales) OVER (PARTITION BY SalesPersonID) AS AvgSalesPersonSales,
    Sales - AVG(Sales) OVER (PARTITION BY SalesPersonID) AS DeviationFromAvg
FROM Sales.Orders;
```

Task 114

Use PERCENT_RANK to show the relative standing of each customer's Score within the whole table.

SOLUTION

```
SELECT
    CustomerID,
    Score,
    PERCENT_RANK() OVER (ORDER BY Score) AS ScorePercentRank
FROM Sales.Customers;
```

Task 115

Find the median Sales value per ProductID using PERCENTILE_CONT.

SOLUTION

```
SELECT DISTINCT
    ProductID,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Sales)
    OVER (PARTITION BY ProductID) AS MedianSales
FROM Sales.Orders;
```

Task 116

Identify gaps of more than 30 days between a customer's consecutive orders (LAG + DATEDIFF).

SOLUTION

```
SELECT *
FROM (
    SELECT
        CustomerID,
        OrderID,
        OrderDate,
        DATEDIFF(DAY, LAG(OrderDate) OVER (PARTITION BY CustomerID ORDER BY OrderDate), OrderDate) AS DaysSincePrevOrder
    FROM Sales.Orders
) AS x
WHERE DaysSincePrevOrder > 30;
```

Task 117

Rank salespeople by total Sales generated, and show only the top 3 (DENSE_RANK in a CTE).

SOLUTION

```
WITH SalesTotals AS (
    SELECT SalesPersonID, SUM(Sales) AS TotalSales
    FROM Sales.Orders
    GROUP BY SalesPersonID
),
Ranked AS (
    SELECT *, DENSE_RANK() OVER (ORDER BY TotalSales DESC) AS SalesRank
    FROM SalesTotals
)
SELECT *
FROM Ranked
WHERE SalesRank <= 3;
```

F. Date & Time Functions

15 tasks in this section

Task 118

Show each order's OrderDate along with the year, month, and day extracted separately.

SOLUTION

```
SELECT
    OrderID,
    OrderDate,
    YEAR(OrderDate) AS OrderYear,
    MONTH(OrderDate) AS OrderMonth,
    DAY(OrderDate) AS OrderDay
FROM Sales.Orders;
```

Task 119

Calculate the number of days each order took to ship (DATEDIFF between OrderDate and ShipDate).

SOLUTION

```
SELECT
    OrderID,
    OrderDate,
    ShipDate,
    DATEDIFF(DAY, OrderDate, ShipDate) AS DaysToShip
FROM Sales.Orders
WHERE ShipDate IS NOT NULL;
```

Task 120

Find all orders shipped more than 7 days after they were placed.

SOLUTION

```
SELECT *
FROM Sales.Orders
WHERE DATEDIFF(DAY, OrderDate, ShipDate) > 7;
```

Task 121

Calculate each employee's current age in years from their BirthDate.

SOLUTION

```
SELECT
    EmployeeID,
    FirstName + ' ' + LastName AS EmployeeName,
    BirthDate,
    DATEDIFF(YEAR, BirthDate, GETDATE())
    - CASE WHEN DATEADD(YEAR, DATEDIFF(YEAR, BirthDate, GETDATE()), BirthDate) > GETDATE()
        THEN 1 ELSE 0 END AS Age
FROM Sales.Employees;
```

Task 122

Find employees who have a birthday (month/day) this month.

SOLUTION

```
SELECT *
FROM Sales.Employees
WHERE MONTH(BirthDate) = MONTH(GETDATE());
```

Task 123

Add 30 days to each order's ShipDate to calculate an estimated warranty expiry date.

SOLUTION

```
SELECT
    OrderID,
    ShipDate,
    DATEADD(DAY, 30, ShipDate) AS WarrantyExpiryDate
FROM Sales.Orders
WHERE ShipDate IS NOT NULL;
```

Task 124

Find all orders placed within the last 90 days relative to today.

SOLUTION

```
SELECT *
FROM Sales.Orders
WHERE OrderDate >= DATEADD(DAY, -90, CAST(GETDATE() AS DATE));
```

Task 125

Format each OrderDate as 'YYYY-MM' to group orders by year and month as text.

SOLUTION

```
SELECT
    OrderID,
    FORMAT(OrderDate, 'yyyy-MM') AS YearMonth
FROM Sales.Orders;
```

Task 126

Find the day of the week name for each OrderDate.

SOLUTION

```
SELECT
    OrderID,
    OrderDate,
    DATENAME(WEEKDAY, OrderDate) AS OrderDayName
FROM Sales.Orders;
```

Task 127

Count how many orders were placed on each day of the week.

SOLUTION

```
SELECT
    DATENAME(WEEKDAY, OrderDate) AS DayOfWeek,
    COUNT(*) AS OrderCount
FROM Sales.Orders
GROUP BY DATENAME(WEEKDAY, OrderDate);
```

Task 128

Find the CreationTime (DATETIME2) truncated to just the date part for each order.

SOLUTION

```
SELECT
    OrderID,
    CreationTime,
    CAST(CreationTime AS DATE) AS CreationDateOnly
FROM Sales.Orders;
```

Task 129

Find orders where CreationTime and OrderDate fall on different calendar dates.

SOLUTION

```
SELECT *
FROM Sales.Orders
WHERE CAST(CreationTime AS DATE) <> OrderDate;
```

Task 130

Calculate the number of full months between each employee's BirthDate and today.

SOLUTION

```
SELECT
    EmployeeID,
    BirthDate,
    DATEDIFF(MONTH, BirthDate, GETDATE()) AS MonthsSinceBirth
FROM Sales.Employees;
```

Task 131

Find the earliest and latest OrderDate in the entire Orders table.

SOLUTION

```
SELECT
    MIN(OrderDate) AS EarliestOrder,
    MAX(OrderDate) AS LatestOrder
FROM Sales.Orders;
```

Task 132

Find all orders placed on a weekend (Saturday or Sunday).

SOLUTION

```
SELECT *
FROM Sales.Orders
WHERE DATENAME(WEEKDAY, OrderDate) IN ('Saturday', 'Sunday');
```

G. String Functions

15 tasks in this section

Task 133

Concatenate each customer's FirstName and LastName into a single FullName column.

SOLUTION

```
SELECT
    CustomerID,
    CONCAT(FirstName, ' ', LastName) AS FullName
FROM Sales.Customers;
```

Task 134

Convert all product names to uppercase.

SOLUTION

```
SELECT ProductID, UPPER(Product) AS ProductUpper FROM Sales.Products;
```

Task 135

Convert all customer countries to lowercase.

SOLUTION

```
SELECT CustomerID, LOWER(Country) AS CountryLower FROM Sales.Customers;
```

Task 136

Find the length (number of characters) of each product's name.

SOLUTION

```
SELECT ProductID, Product, LEN(Product) AS NameLength FROM Sales.Products;
```

Task 137

Extract the first 3 characters of each customer's LastName.

SOLUTION

```
SELECT CustomerID, LastName, LEFT(LastName, 3) AS LastName3 FROM Sales.Customers;
```

Task 138

Extract the last 4 characters of each order's ShipAddress.

SOLUTION

```
SELECT OrderID, ShipAddress, RIGHT(ShipAddress, 4) AS AddressTail FROM Sales.Orders;
```

Task 139

Remove leading and trailing spaces from the Product column (defensive TRIM).

SOLUTION

```
SELECT ProductID, TRIM(Product) AS ProductTrimmed FROM Sales.Products;
```

Task 140

Replace all spaces in the ShipAddress with underscores.

SOLUTION

```
SELECT OrderID, REPLACE(ShipAddress, ' ', '_') AS ShipAddressNoSpaces FROM Sales.Orders;
```

Task 141

Find customers whose FirstName and LastName are the same length.

SOLUTION

```
SELECT *
FROM Sales.Customers
WHERE LEN(FirstName) = LEN(LastName);
```

Task 142

Extract the domain-like substring after the first space in each ShipAddress (using CHARINDEX and SUBSTRING).

SOLUTION

```
SELECT
    OrderID,
    ShipAddress,
    SUBSTRING(ShipAddress, CHARINDEX(' ', ShipAddress) + 1, LEN(ShipAddress)) AS AfterFirstSpace
FROM Sales.Orders
WHERE CHARINDEX(' ', ShipAddress) > 0;
```

Task 143

Build a masked customer code combining the first letter of FirstName, first letter of LastName, and CustomerID.

SOLUTION

```
SELECT
    CustomerID,
    LEFT(FirstName, 1) + LEFT(LastName, 1) + CAST(CustomerID AS VARCHAR(10)) AS CustomerCode
FROM Sales.Customers;
```

Task 144

Find employees whose Department name, once reversed, still reads as a valid comparison (demonstrate REVERSE()).

SOLUTION

```
SELECT EmployeeID, Department, REVERSE(Department) AS DepartmentReversed FROM Sales.Employees;
```

Task 145

Pad each ProductID with leading zeros to create a fixed 5-digit product code (RIGHT + string concatenation).

SOLUTION

```
SELECT
    ProductID,
    RIGHT('00000' + CAST(ProductID AS VARCHAR(10)), 5) AS ProductCode
FROM Sales.Products;
```

Task 146

Find the position of the substring 'Ave' within each ShipAddress, if present.

SOLUTION

```
SELECT
    OrderID,
    ShipAddress,
    CHARINDEX('Ave', ShipAddress) AS AvePosition
FROM Sales.Orders
WHERE CHARINDEX('Ave', ShipAddress) > 0;
```

Task 147

Standardize OrderStatus values by trimming spaces and converting to proper case-insensitive uppercase for comparison.

SOLUTION

```
SELECT OrderID, OrderStatus
FROM Sales.Orders
WHERE UPPER(TRIM(OrderStatus)) = 'DELIVERED';
```

H. CASE Expressions

15 tasks in this section

Task 148

Classify each customer as 'High', 'Medium', or 'Low' based on their Score (≥ 700 High, ≥ 400 Medium, else Low).

SOLUTION

```
SELECT
    CustomerID,
    Score,
    CASE
        WHEN Score >= 700 THEN 'High'
        WHEN Score >= 400 THEN 'Medium'
        ELSE 'Low'
    END AS ScoreSegment
FROM Sales.Customers;
```

Task 149

Label each order as 'Large', 'Medium', or 'Small' based on Quantity (>10 Large, 3-10 Medium, <3 Small).

SOLUTION

```
SELECT
    OrderID,
    Quantity,
    CASE
        WHEN Quantity > 10 THEN 'Large'
        WHEN Quantity BETWEEN 3 AND 10 THEN 'Medium'
        ELSE 'Small'
    END AS OrderSize
FROM Sales.Orders;
```

Task 150

Show each employee's Gender as 'Male' or 'Female' instead of the raw 'M'/'F' code.

SOLUTION

```
SELECT
    EmployeeID,
    CASE Gender
        WHEN 'M' THEN 'Male'
        WHEN 'F' THEN 'Female'
        ELSE 'Unspecified'
    END AS GenderFull
FROM Sales.Employees;
```

Task 151

Flag each order as 'Shipped' or 'Pending' depending on whether ShipDate is populated.

SOLUTION

```
SELECT
    OrderID,
    ShipDate,
    CASE WHEN ShipDate IS NULL THEN 'Pending' ELSE 'Shipped' END AS ShipStatus
FROM Sales.Orders;
```

Task 152

Categorize products into price tiers: 'Premium' (>200), 'Standard' (50-200), 'Budget' (<50).

SOLUTION

```
SELECT
    ProductID,
    Product,
    Price,
    CASE
        WHEN Price > 200 THEN 'Premium'
        WHEN Price BETWEEN 50 AND 200 THEN 'Standard'
        ELSE 'Budget'
    END AS PriceTier
FROM Sales.Products;
```

Task 153

Count how many orders fall into each Quantity-based size category defined via CASE.

SOLUTION

```
SELECT
    CASE
        WHEN Quantity > 10 THEN 'Large'
        WHEN Quantity BETWEEN 3 AND 10 THEN 'Medium'
        ELSE 'Small'
    END AS OrderSize,
    COUNT(*) AS NumOrders
FROM Sales.Orders
GROUP BY
    CASE
        WHEN Quantity > 10 THEN 'Large'
        WHEN Quantity BETWEEN 3 AND 10 THEN 'Medium'
        ELSE 'Small'
    END;
```

Task 154

Classify employees into salary bands ('Band A' >= 8000, 'Band B' >= 5000, 'Band C' otherwise) and count employees per band.

SOLUTION

```
SELECT
    CASE
        WHEN Salary >= 8000 THEN 'Band A'
        WHEN Salary >= 5000 THEN 'Band B'
        ELSE 'Band C'
    END AS SalaryBand,
    COUNT(*) AS EmployeeCount
FROM Sales.Employees
GROUP BY
    CASE
        WHEN Salary >= 8000 THEN 'Band A'
        WHEN Salary >= 5000 THEN 'Band B'
        ELSE 'Band C'
    END;
```

Task 155

Show 'Yes' or 'No' for whether each order's ShipAddress differs from its BillAddress.

SOLUTION

```
SELECT
    OrderID,
    CASE WHEN ShipAddress = BillAddress THEN 'No' ELSE 'Yes' END AS AddressMismatch
FROM Sales.Orders;
```

Task 156

Tag each customer's country as 'Domestic' if it is 'USA', otherwise 'International'.

SOLUTION

```
SELECT
    CustomerID,
    Country,
    CASE WHEN Country = 'USA' THEN 'Domestic' ELSE 'International' END AS MarketType
FROM Sales.Customers;
```

Task 157

Assign a discount percentage per order using CASE based on Quantity thresholds, then calculate the discounted Sales value.

SOLUTION

```
SELECT
    OrderID,
    Quantity,
    Sales,
    CASE
        WHEN Quantity >= 20 THEN 0.15
        WHEN Quantity >= 10 THEN 0.10
        WHEN Quantity >= 5 THEN 0.05
        ELSE 0
    END AS DiscountRate,
    Sales * (1 - CASE
        WHEN Quantity >= 20 THEN 0.15
        WHEN Quantity >= 10 THEN 0.10
        WHEN Quantity >= 5 THEN 0.05
        ELSE 0
    END) AS DiscountedSales
FROM Sales.Orders;
```

Task 158

Show whether each employee is 'Senior' (age 45+) or 'Junior' based on BirthDate.

SOLUTION

```
SELECT
    EmployeeID,
    BirthDate,
    CASE
        WHEN DATEDIFF(YEAR, BirthDate, GETDATE()) >= 45 THEN 'Senior'
        ELSE 'Junior'
    END AS SeniorityLabel
FROM Sales.Employees;
```

Task 159

Label each order's OrderStatus as 'Complete' for Delivered, 'In Progress' for Shipped/Processing, and 'Other' for anything else.

SOLUTION

```
SELECT
    OrderID,
    OrderStatus,
    CASE
        WHEN OrderStatus = 'Delivered' THEN 'Complete'
        WHEN OrderStatus IN ('Shipped', 'Processing') THEN 'In Progress'
        ELSE 'Other'
    END AS StatusGroup
FROM Sales.Orders;
```

Task 160

Compute a bonus-eligibility flag for employees: 'Eligible' if Salary < 6000 AND Department = 'Sales', else 'Not Eligible'.

SOLUTION

```
SELECT
    EmployeeID,
    Department,
    Salary,
    CASE
        WHEN Salary < 6000 AND Department = 'Sales' THEN 'Eligible'
        ELSE 'Not Eligible'
    END AS BonusEligibility
FROM Sales.Employees;
```

Task 161

Show a customer loyalty label combining Score tier and order count tier (nested CASE with a subquery for order count).

SOLUTION

```
SELECT
    c.CustomerID,
    c.Score,
    OrderCounts.OrderCount,
    CASE
        WHEN c.Score >= 700 AND OrderCounts.OrderCount >= 5 THEN 'VIP'
        WHEN c.Score >= 400 THEN 'Regular'
        ELSE 'New/Low Engagement'
    END AS LoyaltyLabel
FROM Sales.Customers AS c
LEFT JOIN (
    SELECT CustomerID, COUNT(*) AS OrderCount
    FROM Sales.Orders
    GROUP BY CustomerID
) AS OrderCounts
ON OrderCounts.CustomerID = c.CustomerID;
```

Task 162

Classify each order's shipping speed as 'Fast' (<=3 days), 'Normal' (4-7 days), or 'Slow' (>7 days).

SOLUTION

```
SELECT
    OrderID,
    DATEDIFF(DAY, OrderDate, ShipDate) AS DaysToShip,
    CASE
        WHEN DATEDIFF(DAY, OrderDate, ShipDate) <= 3 THEN 'Fast'
        WHEN DATEDIFF(DAY, OrderDate, ShipDate) <= 7 THEN 'Normal'
        ELSE 'Slow'
    END AS ShippingSpeed
FROM Sales.Orders
WHERE ShipDate IS NOT NULL;
```

I. Set Operations (UNION / INTERSECT / EXCEPT)

10 tasks in this section

Task 163

Combine CustomerIDs that appear in Orders with CustomerIDs that appear in Orders (dedup with UNION) -- list all distinct customers who ever ordered, from both current and archived orders.

SOLUTION

```
SELECT CustomerID FROM Sales.Orders
UNION
SELECT CustomerID FROM Sales.OrdersArchive;
```

Task 164

List all order rows from both Orders and OrdersArchive, keeping duplicates (UNION ALL), tagged by source.

SOLUTION

```
SELECT *, 'Current' AS SourceTable FROM Sales.Orders
UNION ALL
SELECT *, 'Archive' AS SourceTable FROM Sales.OrdersArchive;
```

Task 165

Find OrderIDs that exist in Orders but NOT in OrdersArchive (EXCEPT).

SOLUTION

```
SELECT OrderID FROM Sales.Orders
EXCEPT
SELECT OrderID FROM Sales.OrdersArchive;
```

Task 166

Find OrderIDs that exist in OrdersArchive but NOT in Orders (EXCEPT, reversed).

SOLUTION

```
SELECT OrderID FROM Sales.OrdersArchive
EXCEPT
SELECT OrderID FROM Sales.Orders;
```

Task 167

Find OrderIDs that exist in BOTH Orders and OrdersArchive (INTERSECT).

SOLUTION

```
SELECT OrderID FROM Sales.Orders
INTERSECT
SELECT OrderID FROM Sales.OrdersArchive;
```

Task 168

Combine a list of high-score customers (Score > 700) with a list of employees earning over 8000, showing a unified Name/Type list.

SOLUTION

```
SELECT FirstName + ' ' + LastName AS FullName, 'Customer' AS SourceType
FROM Sales.Customers
WHERE Score > 700
UNION
SELECT FirstName + ' ' + LastName AS FullName, 'Employee' AS SourceType
FROM Sales.Employees
WHERE Salary > 8000;
```

Task 169

List product categories that exist in Products but have zero matching orders, using EXCEPT against categories that were ordered.

SOLUTION

```
SELECT Category FROM Sales.Products
EXCEPT
SELECT p.Category
FROM Sales.Products AS p
JOIN Sales.Orders AS o ON o.ProductID = p.ProductID;
```

Task 170

Find CustomerIDs who ordered in the current Orders table AND also have a matching row in OrdersArchive (INTERSECT with a twist -- customer-level).

SOLUTION

```
SELECT CustomerID FROM Sales.Orders
INTERSECT
SELECT CustomerID FROM Sales.OrdersArchive;
```

Task 171

Build a combined list of all distinct ShipAddress values used in either Orders or OrdersArchive.

SOLUTION

```
SELECT ShipAddress FROM Sales.Orders
UNION
SELECT ShipAddress FROM Sales.OrdersArchive;
```

Task 172

Find SalesPersonIDs who generated orders in OrdersArchive but who have NO orders at all in the current Orders table (EXCEPT).

SOLUTION

```
SELECT SalesPersonID FROM Sales.OrdersArchive
EXCEPT
SELECT SalesPersonID FROM Sales.Orders;
```

J. Common Table Expressions (CTEs)

10 tasks in this section

Task 173

Use a CTE to calculate total Sales per customer, then filter to customers above 5000 in the outer query.

SOLUTION

```
WITH CustomerSales AS (  
    SELECT CustomerID, SUM(Sales) AS TotalSales  
    FROM Sales.Orders  
    GROUP BY CustomerID  
)  
SELECT *  
FROM CustomerSales  
WHERE TotalSales > 5000  
ORDER BY TotalSales DESC;
```

Task 174

Use a CTE to build a full employee-management hierarchy chain (recursive CTE) starting from top-level employees (no manager).

SOLUTION

```
WITH EmployeeHierarchy AS (  
    -- Anchor: top-level employees  
    SELECT EmployeeID, FirstName, LastName, ManagerID, 1 AS HierarchyLevel  
    FROM Sales.Employees  
    WHERE ManagerID IS NULL  
  
    UNION ALL  
  
    -- Recursive: employees reporting to someone already in the hierarchy  
    SELECT e.EmployeeID, e.FirstName, e.LastName, e.ManagerID, eh.HierarchyLevel + 1  
    FROM Sales.Employees AS e  
    JOIN EmployeeHierarchy AS eh  
    ON e.ManagerID = eh.EmployeeID  
)  
SELECT *  
FROM EmployeeHierarchy  
ORDER BY HierarchyLevel, EmployeeID;
```

Task 175

Use a CTE to compute average Price per Category, then join it back to Products to flag above-average items.

SOLUTION

```
WITH CategoryAvg AS (
    SELECT Category, AVG(Price) AS AvgCategoryPrice
    FROM Sales.Products
    GROUP BY Category
)
SELECT
    p.ProductID,
    p.Product,
    p.Category,
    p.Price,
    ca.AvgCategoryPrice,
    CASE WHEN p.Price > ca.AvgCategoryPrice THEN 'Above Avg' ELSE 'At/Below Avg' END AS Comparison
FROM Sales.Products AS p
JOIN CategoryAvg AS ca
    ON ca.Category = p.Category;
```

Task 176

Use multiple CTEs to first calculate order counts per customer, then join with customer details to show only customers with 0 orders.

SOLUTION

```
WITH OrderCounts AS (
    SELECT CustomerID, COUNT(*) AS OrderCount
    FROM Sales.Orders
    GROUP BY CustomerID
),
CustomersWithCounts AS (
    SELECT c.CustomerID, c.FirstName, c.LastName, ISNULL(oc.OrderCount, 0) AS OrderCount
    FROM Sales.Customers AS c
    LEFT JOIN OrderCounts AS oc
        ON oc.CustomerID = c.CustomerID
)
SELECT *
FROM CustomersWithCounts
WHERE OrderCount = 0;
```

Task 177

Use a CTE with ROW_NUMBER to remove duplicate OrderID rows (keep only the first occurrence) when combining Orders and OrdersArchive.

SOLUTION

```
WITH Combined AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY OrderID ORDER BY OrderDate) AS rn
    FROM (
        SELECT * FROM Sales.Orders
        UNION ALL
        SELECT * FROM Sales.OrdersArchive
    ) AS AllOrders
)
SELECT *
FROM Combined
WHERE rn = 1;
```

Task 178

Use a recursive CTE to count how many direct and indirect reports each manager has.

SOLUTION

```
WITH ReportChain AS (  
    SELECT EmployeeID, ManagerID, EmployeeID AS RootEmployee  
    FROM Sales.Employees  
  
    UNION ALL  
  
    SELECT e.EmployeeID, e.ManagerID, rc.RootEmployee  
    FROM Sales.Employees AS e  
    JOIN ReportChain AS rc  
        ON e.ManagerID = rc.EmployeeID  
)  
SELECT ManagerID, COUNT(*) AS TotalReports  
FROM ReportChain  
WHERE ManagerID IS NOT NULL  
GROUP BY ManagerID;
```

Task 179

Use a CTE to rank products by total revenue, then select only ranks 4 through 6 (a 'second tier' report).

SOLUTION

```
WITH ProductRevenue AS (  
    SELECT ProductID, SUM(Sales) AS TotalRevenue  
    FROM Sales.Orders  
    GROUP BY ProductID  
)  
,  
RankedRevenue AS (  
    SELECT *, RANK() OVER (ORDER BY TotalRevenue DESC) AS RevenueRank  
    FROM ProductRevenue  
)  
SELECT *  
FROM RankedRevenue  
WHERE RevenueRank BETWEEN 4 AND 6;
```

Task 180

Use a CTE to calculate monthly sales totals, then compute month-over-month growth using LAG.

SOLUTION

```
WITH MonthlySales AS (  
    SELECT  
        YEAR(OrderDate) AS OrderYear,  
        MONTH(OrderDate) AS OrderMonth,  
        SUM(Sales) AS TotalSales  
    FROM Sales.Orders  
    GROUP BY YEAR(OrderDate), MONTH(OrderDate)  
)  
SELECT  
    OrderYear,  
    OrderMonth,  
    TotalSales,  
    TotalSales - LAG(TotalSales) OVER (ORDER BY OrderYear, OrderMonth) AS GrowthVsPrevMonth  
FROM MonthlySales  
ORDER BY OrderYear, OrderMonth;
```

Task 181

Use a CTE to isolate 'high value' orders (Sales > 2000), then join back to Customers to show who placed them.

SOLUTION

```
WITH HighValueOrders AS (  
    SELECT * FROM Sales.Orders WHERE Sales > 2000  
)  
SELECT  
    c.FirstName + ' ' + c.LastName AS CustomerName,  
    h.OrderID,  
    h.Sales  
FROM HighValueOrders AS h  
JOIN Sales.Customers AS c  
    ON c.CustomerID = h.CustomerID;
```

Task 182

Use a CTE to find each customer's average order value, then flag customers whose latest order was above their own average.

SOLUTION

```
WITH CustomerAvg AS (  
    SELECT CustomerID, AVG(Sales) AS AvgSales  
    FROM Sales.Orders  
    GROUP BY CustomerID  
)  
,  
LatestOrder AS (  
    SELECT o.*,  
        ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY OrderDate DESC) AS rn  
    FROM Sales.Orders AS o  
)  
SELECT  
    lo.CustomerID,  
    lo.OrderID,  
    lo.Sales AS LatestOrderSales,  
    ca.AvgSales,  
    CASE WHEN lo.Sales > ca.AvgSales THEN 'Above Average' ELSE 'At/Below Average' END AS Comparison  
FROM LatestOrder AS lo  
JOIN CustomerAvg AS ca ON ca.CustomerID = lo.CustomerID  
WHERE lo.rn = 1;
```

K. Self Joins

5 tasks in this section

Task 183

List each employee with their manager's Department (may differ from their own).

SOLUTION

```
SELECT
    e.FirstName + ' ' + e.LastName AS EmployeeName,
    e.Department AS EmployeeDept,
    m.Department AS ManagerDept
FROM Sales.Employees AS e
JOIN Sales.Employees AS m
    ON e.ManagerID = m.EmployeeID;
```

Task 184

Find pairs of employees who share the same manager.

SOLUTION

```
SELECT
    e1.FirstName + ' ' + e1.LastName AS Employee1,
    e2.FirstName + ' ' + e2.LastName AS Employee2,
    e1.ManagerID
FROM Sales.Employees AS e1
JOIN Sales.Employees AS e2
    ON e1.ManagerID = e2.ManagerID
    AND e1.EmployeeID < e2.EmployeeID
WHERE e1.ManagerID IS NOT NULL;
```

Task 185

Find employees who earn the same Salary as at least one other employee (self join on Salary).

SOLUTION

```
SELECT DISTINCT e1.*
FROM Sales.Employees AS e1
JOIN Sales.Employees AS e2
    ON e1.Salary = e2.Salary
    AND e1.EmployeeID <> e2.EmployeeID;
```

Task 186

For each manager, list the salary gap between the manager and each of their direct reports.

SOLUTION

```
SELECT
    m.FirstName + ' ' + m.LastName AS ManagerName,
    e.FirstName + ' ' + e.LastName AS EmployeeName,
    m.Salary - e.Salary AS SalaryGap
FROM Sales.Employees AS e
JOIN Sales.Employees AS m
    ON e.ManagerID = m.EmployeeID;
```

Task 187

Find two customers from the same country whose Scores differ by less than 10 points (self join with a tolerance).

SOLUTION

```
SELECT
    c1.CustomerID AS Customer1,
    c2.CustomerID AS Customer2,
    c1.Country,
    c1.Score AS Score1,
    c2.Score AS Score2
FROM Sales.Customers AS c1
JOIN Sales.Customers AS c2
    ON c1.Country = c2.Country
    AND c1.CustomerID < c2.CustomerID
WHERE ABS(c1.Score - c2.Score) < 10;
```

L. NULL Handling

10 tasks in this section

Task 188

Replace NULL customer Scores with 0 using ISNULL.

SOLUTION

```
SELECT CustomerID, ISNULL(Score, 0) AS ScoreFilled FROM Sales.Customers;
```

Task 189

Replace NULL ManagerID with the text 'No Manager' using COALESCE (with type casting).

SOLUTION

```
SELECT
    EmployeeID,
    COALESCE(CAST(ManagerID AS VARCHAR(10)), 'No Manager') AS ManagerDisplay
FROM Sales.Employees;
```

Task 190

Use COALESCE to pick the first non-NULL value among ShipDate, then OrderDate, as an 'EffectiveDate' column.

SOLUTION

```
SELECT
    OrderID,
    COALESCE(ShipDate, OrderDate) AS EffectiveDate
FROM Sales.Orders;
```

Task 191

Find all columns in Orders where Sales equals Quantity (edge case) using NULLIF to avoid division by zero elsewhere.

SOLUTION

```
SELECT
    OrderID,
    Sales,
    Quantity,
    Sales / NULLIF(Quantity, 0) AS PricePerUnit
FROM Sales.Orders;
```

Task 192

Count how many customers have a NULL Score versus a non-NULL Score.

SOLUTION

```
SELECT
    SUM(CASE WHEN Score IS NULL THEN 1 ELSE 0 END) AS NullScores,
    SUM(CASE WHEN Score IS NOT NULL THEN 1 ELSE 0 END) AS NonNullScores
FROM Sales.Customers;
```

Task 193

Find orders where ShipAddress is NULL but BillAddress is populated.

SOLUTION

```
SELECT *
FROM Sales.Orders
WHERE ShipAddress IS NULL AND BillAddress IS NOT NULL;
```

Task 194

Calculate average Score treating NULLs as 0 versus the default AVG behavior (which ignores NULLs).

SOLUTION

```
SELECT
    AVG(Score) AS AvgIgnoringNulls,
    AVG(ISNULL(Score, 0)) AS AvgTreatingNullsAsZero
FROM Sales.Customers;
```

Task 195

List employees, substituting 'Unknown' for a NULL Department.

SOLUTION

```
SELECT EmployeeID, ISNULL(Department, 'Unknown') AS DepartmentFilled FROM Sales.Employees;
```

Task 196

Find orders where the calculated DaysToShip would error due to a NULL ShipDate, and instead show 'Not Shipped'.

SOLUTION

```
SELECT
    OrderID,
    CASE
        WHEN ShipDate IS NULL THEN 'Not Shipped'
        ELSE CAST(DATEDIFF(DAY, OrderDate, ShipDate) AS VARCHAR(10))
    END AS ShippingStatus
FROM Sales.Orders;
```

Task 197

Use ISNULL to default missing SalesPersonID values to 0 when summarizing total Sales per salesperson.

SOLUTION

```
SELECT
    ISNULL(SalesPersonID, 0) AS SalesPersonID,
    SUM(Sales) AS TotalSales
FROM Sales.Orders
GROUP BY ISNULL(SalesPersonID, 0);
```

M. TOP-N & Pagination

10 tasks in this section

Task 198

Find the top 5 customers by Score.

SOLUTION

```
SELECT TOP 5 *  
FROM Sales.Customers  
ORDER BY Score DESC;
```

Task 199

Find the 3 most expensive products.

SOLUTION

```
SELECT TOP 3 *  
FROM Sales.Products  
ORDER BY Price DESC;
```

Task 200

Find the top 10% of orders by Sales value (TOP PERCENT).

SOLUTION

```
SELECT TOP 10 PERCENT *  
FROM Sales.Orders  
ORDER BY Sales DESC;
```

Task 201

Find the top 5 highest-paid employees, including ties (TOP WITH TIES).

SOLUTION

```
SELECT TOP 5 WITH TIES *  
FROM Sales.Employees  
ORDER BY Salary DESC;
```

Task 202

Return the second page of customers (10 per page) ordered by CustomerID, using OFFSET-FETCH.

SOLUTION

```
SELECT *  
FROM Sales.Customers  
ORDER BY CustomerID  
OFFSET 10 ROWS FETCH NEXT 10 ROWS ONLY;
```

Task 203

Find the 3 lowest-selling products by total Quantity ordered.

SOLUTION

```
SELECT TOP 3
    ProductID,
    SUM(Quantity) AS TotalQuantity
FROM Sales.Orders
GROUP BY ProductID
ORDER BY TotalQuantity ASC;
```

Task 204

Find the single most recent order placed.

SOLUTION

```
SELECT TOP 1 *
FROM Sales.Orders
ORDER BY OrderDate DESC;
```

Task 205

Pageinate all orders 20 rows at a time and return page 3, ordered by OrderID.

SOLUTION

```
SELECT *
FROM Sales.Orders
ORDER BY OrderID
OFFSET 40 ROWS FETCH NEXT 20 ROWS ONLY;
```

Task 206

Find the top 5 customers by total Sales (requires aggregation before TOP).

SOLUTION

```
SELECT TOP 5
    CustomerID,
    SUM(Sales) AS TotalSales
FROM Sales.Orders
GROUP BY CustomerID
ORDER BY TotalSales DESC;
```

Task 207

Find the bottom 3 departments by average Salary.

SOLUTION

```
SELECT TOP 3
    Department,
    AVG(Salary) AS AvgSalary
FROM Sales.Employees
GROUP BY Department
ORDER BY AvgSalary ASC;
```

N. Miscellaneous

10 tasks in this section

Task 208

Create a view that exposes each order with its customer name and product name for easy reuse.

SOLUTION

```
CREATE VIEW Sales.vw_OrderDetails AS
SELECT
    o.OrderID,
    c.FirstName + ' ' + c.LastName AS CustomerName,
    p.Product,
    o.Quantity,
    o.Sales,
    o.OrderDate
FROM Sales.Orders AS o
JOIN Sales.Customers AS c ON c.CustomerID = o.CustomerID
JOIN Sales.Products AS p ON p.ProductID = o.ProductID;
GO

-- Usage:
-- SELECT * FROM Sales.vw_OrderDetails WHERE OrderDate >= '2024-01-01';
```

Task 209

Create a view summarizing total Sales and order count per customer.

SOLUTION

```
CREATE VIEW Sales.vw_CustomerSummary AS
SELECT
    c.CustomerID,
    c.FirstName + ' ' + c.LastName AS CustomerName,
    COUNT(o.OrderID) AS TotalOrders,
    ISNULL(SUM(o.Sales), 0) AS TotalSales
FROM Sales.Customers AS c
LEFT JOIN Sales.Orders AS o
    ON o.CustomerID = c.CustomerID
GROUP BY c.CustomerID, c.FirstName, c.LastName;
GO
```

Task 210

List all distinct Country values present in the Customers table.

SOLUTION

```
SELECT DISTINCT Country FROM Sales.Customers;
```

Task 211

List all distinct combinations of Department and Gender in the Employees table.

SOLUTION

```
SELECT DISTINCT Department, Gender FROM Sales.Employees;
```

Task 212

Add a computed-style column in a query that calculates Sales per unit (Sales / Quantity) for every order.

SOLUTION

```
SELECT
    OrderID,
    Sales,
    Quantity,
    CAST(Sales AS FLOAT) / NULLIF(Quantity, 0) AS SalesPerUnit
FROM Sales.Orders;
```

Task 213

Find duplicate ShipAddress values that appear on more than one order.

SOLUTION

```
SELECT ShipAddress, COUNT(*) AS Occurrences
FROM Sales.Orders
GROUP BY ShipAddress
HAVING COUNT(*) > 1;
```

Task 214

Use SELECT INTO to create a backup snapshot table of all 2023 orders.

SOLUTION

```
SELECT *
INTO Sales.Orders_2023_Backup
FROM Sales.Orders
WHERE YEAR(OrderDate) = 2023;
```

Task 215

Write a query that returns TRUE/FALSE style output (1/0) for whether each customer's Score is above the median score, using a CTE for the median.

SOLUTION

```
WITH MedianScore AS (
    SELECT DISTINCT
        PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Score) OVER () AS MedScore
    FROM Sales.Customers
)
SELECT
    c.CustomerID,
    c.Score,
    CASE WHEN c.Score > m.MedScore THEN 1 ELSE 0 END AS AboveMedian
FROM Sales.Customers AS c
CROSS JOIN MedianScore AS m;
```

Task 216

Use CROSS JOIN to generate every possible Country/Category combination (for a coverage matrix report).

SOLUTION

```
SELECT DISTINCT
    c.Country,
    p.Category
FROM Sales.Customers AS c
CROSS JOIN Sales.Products AS p
ORDER BY c.Country, p.Category;
```

Task 217

Write a query using EXCEPT to find customers who exist in the Customers table but whose CustomerID never appears as a SalesPersonID (sanity check that customers and employees don't overlap in role).

SOLUTION

```
SELECT CustomerID FROM Sales.Customers
EXCEPT
SELECT SalesPersonID FROM Sales.Orders WHERE SalesPersonID IS NOT NULL;
```
